PYTHON MODULE 03

Here's a **simple, exam-like but detailed explanation** of all the listed topics under **Modules in Python**, with subtopics, properties, and usage—all written in a clean and easy-to-understand manner 👇

🌟 Modules in Python

🔷 1. Introduction to Modules and Packages in Python

- **Module**:
  A module is a file that contains **Python code** — functions, classes, or variables — saved with a .py extension.
  Modules help in **code reuse**, **organization**, and **maintenance**.

📌 Why use modules?

- Avoid code repetition
- Improve readability and structure
- Make debugging easier
- Divide large programs into smaller parts

- **Package**:
  A package is a **collection of related modules**, organized in directories, and includes an __init__.py file (can be empty).

□ Package = Folder + __init__.py + Modules inside

🔷 2. Creation of a Module

To create a module:

1. Write your code (functions, classes, variables)
2. Save the file with a .py extension

Example: my_module.py

```
def greet(name):
    return f"Hello, {name}!"
```

Now my_module.py is a custom module you can import and use anywhere.

🔷 3. Importing Modules

There are several ways to import modules in Python:

✅ Import entire module

```
import my_module

print(my_module.greet("Conan"))
```

✅ Import specific function/class

```
from my_module import greet

print(greet("Conan"))
```

✅ Import with alias (nickname)

```
import my_module as mm

print(mm.greet("Conan"))
```

✅ **Import all contents (not recommended)**

```
from my_module import *
```

🔷 4. Uses of Standard Modules in Python

Python comes with many **built-in standard modules**. Here are some commonly used ones:

📐 Math Module

Used for **mathematical operations**.

✅ **Common Functions:**

- math.sqrt(x) → square root
- math.pow(x, y) → x raised to power y
- math.factorial(x)
- math.pi, math.e → constants

Example:

```
import math

print(math.sqrt(25))  # Output: 5.0
```

□ SymPy Module

Used for **symbolic mathematics** like algebra, calculus.

✅ **Features:**

- Algebraic equations
- Simplification
- Derivatives & integrals
- Solving equations symbolically

Example:

```
from sympy import symbols, solve
x = symbols('x')
equation = x**2 - 4
print(solve(equation))  # Output: [-2, 2]
```

🗀 os Module

Used to interact with the **Operating System**.

✅ **Common Functions:**

- os.getcwd() → Get current directory
- os.mkdir("new") → Make new folder
- os.remove("file.txt") → Remove file
- os.listdir() → List all files in directory

Example:

```
import os
```

```
print(os.getcwd())  # Shows current working directory
```

💻 sys Module

Used to interact with the **Python interpreter**.

✅ Common Functions:

- sys.argv → Command-line arguments
- sys.exit() → Exit program
- sys.path → List of paths for module search
- sys.version → Python version info

Example:

```
import sys
print(sys.version)
```

🛠 py_compile Module

Used to **compile Python files** to bytecode .pyc files.

✅ Why compile?

- Faster execution
- Secures source code from direct editing

Example:

```
import py_compile
py_compile.compile('my_module.py')
```

Creates __pycache__ folder with compiled file inside.

🔷 5. Packages

✅ **Definition**:
A package is a **folder that contains multiple modules** and a special __init__.py file.

✅ **Creating a Package:**

my_package/

├── __init__.py

├── module1.py

└── module2.py

✅ **Using the Package:**

```
from my_package import module1

module1.function_name()
```

📝 The __init__.py file is required (even if empty) so Python treats the folder as a package.

✅ **Summary:**

Topic	Description
Module	A single .py file with reusable code
Package	A collection of modules inside a folder

Topic	Description
Importing	import, from, as keywords used to bring module code into your program
math module	For numeric and mathematical operations
sympy module	For algebra and calculus in symbolic form
os module	Interact with files and folders
sys module	Interact with Python interpreter
py_compile	Convert .py files to .pyc (bytecode)
Package usage	Requires __init__.py and can include many related modules

---

Let me know if you want quick MCQs, short answers, or a practice sheet for revision! 😊